Google Workspace Adoption, Change Management & Training Program (Enterprise)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary 4

Solution Overview 4

What this is . . . 4

How it works . . . 5

Reference design . . . 5

Benefits . . . 6

Limitations & boundaries . . . 6

Deployment model . . . 7

Requirements 7

1. An approved migration / rollout plan with a wave schedule . . . 7

2. Executive sponsorship and a change mandate . . . 7

3. Communication channels to reach every user . . . 8

4. Training delivery capacity and an LMS / completion source . . . 8

5. Google Workspace admin roles for adoption **measurement** (read-only) . . . 8

6. A champion-network mandate (time allocation) . . . 9

7. Gemini licensing clarity (per cohort) . . . 9

Global Requirements — Google Workspace & Gemini Solutions 10

1. Google Workspace / Cloud Identity tenant . . . 10

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 10

3. DNS control for the routed domains . . . 10

4. Identity & provisioning source of truth . . . 11

5. A pilot scope (OU / group) and change control . . . 11

6. Security, compliance & governance stakeholders . . . 11

Implementation 12

Phase 1 — Assess . . . 12

Phase 2 — Design . . . 12

Phase 3 — Configure . . . 13

Phase 4 — Pilot (`Adoption Pilot – Wave 0 – Pilot`) . . . 14

Phase 5 — Validate (pilot pass/fail gate) . . . 14

Phase 6 — Expand (production waves) . . . 15

Testing & Validation 15

T1 — Read-only measurement access works (`adoption_config_check.py`) . . . 15

T2 — Adoption baseline captured (`adoption_metrics_report.py`) . . . 15

T3 — Champion coverage meets ratio (`champion_roster_planner.py`) . . . 16

T4 — Training completion gate met (`training_completion_tracker.py`) . . . 16

T5 — Hypercare is staffed and themes are understood (`hypercare_theme_analyzer.py`) 16
T6 — Adoption is trending up vs. baseline (`adoption_metrics_report.py`) 16
Pilot validation summary (the scale-out gate) 17
Troubleshooting 17
T1 — `adoption_config_check.py` reports FAIL 17
T2 — `adoption_metrics_report.py` returns empty / zeros 17
T3 — A wave is flagged `UNDER_RATIO` (`champion_roster_planner.py`) 18
T4 — Training gate fails (`training_completion_tracker.py` → `GATE: FAIL`) 18
T5 — Hypercare overwhelmed or `STAFFING: SHORT` 18
T6 — Adoption flat or below baseline (`TREND: FLAT`/`DOWN`) 18
Program-level issues 19
Best Practices 19
Program & change management 19
Champion network 19
Training 20
Hypercare & support 20
Measurement & data hygiene 20
Sustaining adoption 21
Deliverables Summary 21
Documentation 21
Python tools (`scripts/`) — read-only or `--dry-run` by default 21
Appendix A — PowerShell Scripts 23

Executive Summary

The problem. A technically flawless migration of ~150,000 users from Microsoft 365 to Google Workspace still fails if people don't change how they work. Users forward to old inboxes, save to shadow file shares, meet in the wrong tool, and never touch Gemini. The migration is *delivered* but its value — the license spend, the productivity case, the platform decision — is not *realized*.

The outcome. This solution is the **adoption, change-management, and training program** that turns the cutover into measured behavior change: users trained by role, supported at go-live, and actively using the new tools — with the usage data to prove it. It is the human side of the transformation, run as a repeatable, wave-aligned program rather than a one-shot launch email.

The approach. Every migration wave runs the same OCM cycle — **awareness** → **training** → **reinforcement** — carried locally by a **champion network** (respected peers, ~1 per 50–100 users) and backed by a visible **executive sponsor**. Role-based learning paths deliver just-in-time skills for Gmail, Drive, Docs, Meet, and Chat (and **Gemini where the cohort is licensed and features are enabled**), with completion **gating go-live**. A staffed **hypercare/floorwalker** model absorbs the first-days support surge, and **adoption metrics** — computed read-only from the Google Admin SDK Reports API — show whether each cohort's usage is actually climbing, driving the next wave. The whole cycle is proven **pilot-first** on a friendly Wave 0 before it scales.

Why it is safe and governed. The program follows the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** with explicit pass/fail gates (training delivered, hypercare staffed, adoption trending up) and a documented **hold/rollback** for any wave that fails. Its five Python tools are **read-only or dry-run by default**, authenticate via Application Default Credentials, and contain **no hardcoded credentials or service-account keys** — the program measures adoption without ever writing user data. Messaging tone, mandatory-training policy, and Gemini feature availability are verified with the organization's HR/Comms and in-tenant Admin console rather than assumed.

The bottom line. A Google Workspace move that *sticks*: champions engaged, users trained by role, hypercare staffed, and adoption measured and climbing — the human side of a 150k-user transformation, done right, at a fraction of traditional consulting cost.

Solution Overview

What this is

This solution is the **human side** of a large-scale Microsoft 365 → Google Workspace transformation: the **adoption, organizational-change-management (OCM), and training program** that makes a technically successful migration actually change how ~150,000 users work. A flawless mail, Drive, and Chat cutover still fails if people keep working the way they always did — forwarding to old inboxes, saving to shadow file shares, scheduling meetings in the wrong tool, or never touching Gemini. This program engineers the behavior change that turns a *cutover* into measured *adoption*.

It is deliberately lighter on `gcloud` provisioning than the platform-build solutions in this category and heavier on program design — stakeholder and champion networks, a structured awareness → training → reinforcement arc, role-based learning paths, a hypercare/floorwalker support model, and an adoption-measurement system. It still ships **five Python tools** that instrument and operate the program

against real Google APIs (Admin SDK Reports, Directory) — read-only or dry-run by default.

How it works

The program runs **wave-aligned to the migration**. Each migration wave (a set of users/OUs cut over together) gets the same repeatable OCM cycle so that no cohort lands on Google Workspace un-prepared and un-supported:

1. **Awareness** — before a wave cuts over, the comms campaign tells that cohort *what is changing,

when, why, and where to get help*. Executive sponsorship and the champion network carry the message locally so it is not "an email from IT".

1. **Training** — role-based learning paths deliver *just-in-time* skills (Gmail, Drive, Docs/Sheets/

Slides, Meet, Chat, and **Gemini where the wave is licensed**) sized to what each role actually does, with completion tracked as a go-live gate.

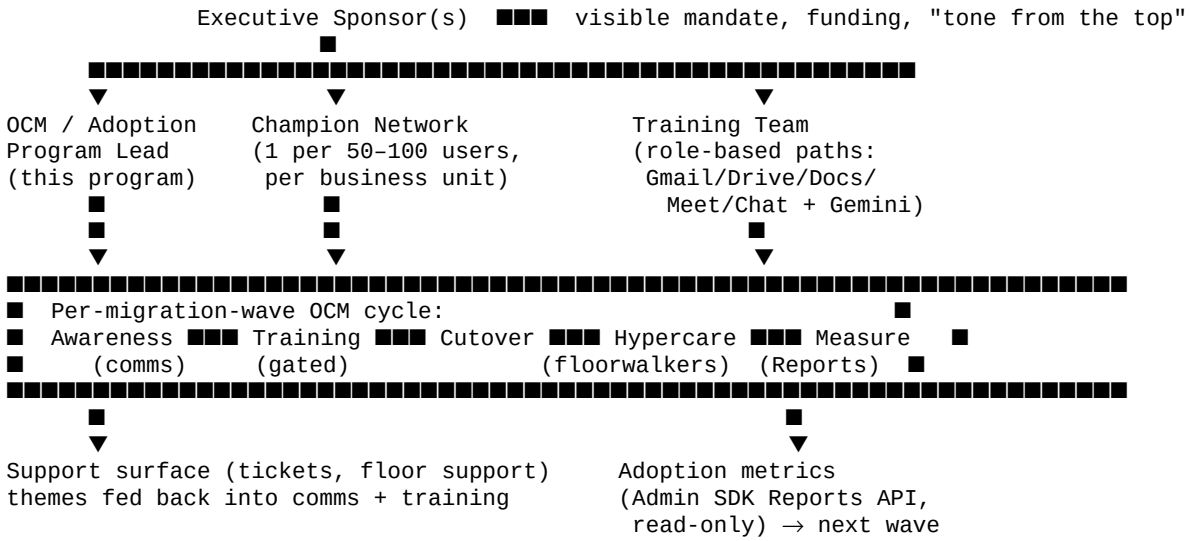
1. **Reinforcement** — after cutover, **hypercare** (a staffed support surface plus on-site or virtual

floorwalkers) resolves the flood of "how do I…?" moments in the first days, and adoption metrics tell you whether usage is actually climbing or whether a cohort needs another push.

The **champion network** is the load-bearing structure across all three phases: a distributed set of respected peers (roughly 1 per 50–100 users, tuned per org) who translate change locally, run floor-level support, feed themes back to the program, and model the new tools. Champions are recruited, trained ahead of their business unit, equipped with content, and recognized.

Adoption measurement closes the loop. The Google **Admin SDK Reports API** (Workspace usage reports) and Directory data are read (never written) to compute per-wave, per-app active-usage trends; training-completion is tracked from the LMS/roster; hypercare ticket themes are analyzed to find where training or comms fell short. Those signals drive the next wave — this is a learning program, not a one-shot launch.

Reference design



Named artifacts this program creates and reuses (consistent convention `<Purpose> – <Scope> – <Stage>`):

- `Adoption Program – Enterprise – v1` — the master OCM plan.
- `Champion Network – Enterprise – v1` — the champion roster/model, with per-BU sub-rosters (`Champion Cohort – <BU> – Wave N`).
- `Comms Plan – Enterprise – v1` — the awareness campaign calendar.
- `Learning Path – <Role> – v1` — the role-based training paths (e.g. `Learning Path – Frontline – v1`).
- `Hypercare Model – <Wave/Region> – Golive` — the staffed support/floorwalker plan per wave.
- `Adoption Metrics – Enterprise – v1` — the measurement plan and dashboard definition.
- `Adoption Pilot – Wave 0 – Pilot` — the pilot cohort that proves the whole cycle before scale-out.

Benefits

- **Migration value is realized, not just delivered** — usage of the target tools actually rises, so the license spend and the migration effort pay off.
- **Repeatable per-wave playbook** — every cohort gets the same prepared, supported experience; quality does not degrade as the program scales to 150k users.
- **Local, trusted change agents** — the champion network makes change peer-led, dramatically cheaper and stickier than centrally-delivered support alone.
- **Evidence-driven** — real Workspace usage data (Reports API) and completion/ticket data show where to intervene, and prove ROI to the sponsor.
- **Safe by construction** — the tooling is read-only or dry-run by default; the program never changes user data to "measure" it.

Limitations & boundaries

- This is the **adoption/OCM/training** layer. It **assumes** a migration/coexistence plan and the underlying Workspace tenant, identity, and licensing exist (see Requirements and the category `global-requirements-google.md`) — it does not perform the mail/Drive/Chat data migration itself.
- **Gemini** content is included **only for waves/cohorts that are licensed** for Gemini (Gemini for Workspace / Gemini Enterprise add-on). Gemini feature availability is **version- and rollout- dependent** — confirm which features (e.g. Gems, Help me write, side-panel in specific apps) are enabled for your cohorts in the Admin console before building the Gemini learning path; **verify in your tenant** rather than assuming.

• **Adoption targets and cadence** are org-specific; the numbers here (champion ratios, hypercare staffing, metric thresholds) are **defaults to tune**, not universal constants.

• **HR/comms/legal ownership** — messaging tone, mandatory-training policy, and any workforce-relations considerations are owned by the organization's HR/Comms/Legal functions; this program supplies the method and the content, not those determinations.

Deployment model

The program follows the house model **Assess** → **(Design** →**) Configure** → **Pilot** → **Validate** → **Expand**, **pilot-first**: the full OCM cycle is proven on `Adoption Pilot – Wave 0 – Pilot` (a small, friendly cohort) with explicit pass/fail gates before it is scaled to production waves, and every wave has a documented **rollback/hold** (pause the wave's cutover, extend hypercare, re-run comms/training) rather than pushing an unprepared cohort live.

Requirements

These are the **solution-specific** prerequisites for the adoption, change-management, and training program. They build **on top of** the shared category requirements in `global-requirements-google.md` (Workspace/Cloud Identity tenant with a super-admin, the source M365/Entra environment, DNS control, an identity source of truth, a pilot OU/group with change control, and engaged security/compliance stakeholders) — those are assumed in place and are **not** repeated here. Verify each item below before starting the implementation steps.

1. An approved migration / rollout plan with a wave schedule

• **Description:** A documented migration plan that defines the **waves** (which OUs/groups/business units cut over from M365 to Google Workspace, in what order, on what dates) and the cutover method (big-bang per wave vs. coexistence). The adoption program aligns every OCM cycle to these waves.

• **Why required:** Awareness, training, and hypercare are all timed **relative to each cohort's cutover date**. Without a wave schedule there is nothing to align to and cohorts will land unprepared.

• **How to verify:** Obtain the current wave schedule (cohort → date → user count) from the migration program. Confirm it maps to real OUs/groups in the Google Admin console (**admin.google.com** → **Directory** → **Organizational units / Groups**).

• **How to implement if missing:** Work with the migration program to define waves and dates before building the OCM calendar; at minimum define `Adoption Pilot – Wave 0 – Pilot` and Wave 1.

2. Executive sponsorship and a change mandate

• **Description:** A named **executive sponsor** (or sponsor coalition) who will visibly back the

change — fund the program, send "tone from the top" communications, and make training expectations clear.

• **Why required:** Adoption programs succeed or fail on visible leadership. Champions, mandatory-training expectations, and comms all draw authority from the sponsor.

• **How to verify:** Confirm a named sponsor has agreed in writing to the mandate and to send at least the launch and per-wave leadership messages.

• **How to implement if missing:** Escalate through the migration steering committee to secure a sponsor before launch; a program without one should not proceed to Expand.

3. Communication channels to reach every user

• **Description:** Working channels to reach all in-scope users **before and after** cutover — email (both the M365 source mailbox pre-cutover and the new Google mailbox post-cutover), intranet, digital signage, Chat/Spaces, and manager cascades.

• **Why required:** Awareness must reach users on whatever tool they still use. Pre-cutover comms must land in the **old** mailbox; post-cutover in the **new** one.

• **How to verify:** Confirm you can publish to each channel and that a distribution mechanism exists for the wave cohorts (e.g. dynamic groups per wave).

• **How to implement if missing:** Establish the channels and the per-wave distribution lists; create Google Groups per wave if needed (**Directory** → **Groups**).

4. Training delivery capacity and an LMS / completion source

• **Description:** The capacity to **deliver** role-based training (live virtual sessions, recorded micro-learning, quick-reference guides) and a **system of record for completion** — an LMS, or at minimum a tracked roster — so completion can gate go-live.

• **Why required:** Training completion is a **go-live gate**; you must be able to record and report who completed which `Learning Path – <Role> – v1`.

• **How to verify:** Confirm the LMS/roster exists and that completion can be exported (CSV/API) for the completion tracker tool.

• **How to implement if missing:** Stand up a completion roster (even a governed spreadsheet) and a content-hosting location before Wave 0.

5. Google Workspace admin roles for adoption measurement (read-only)

• **Description:** A least-privilege admin role (or delegated service account with domain-wide delegation) that can **read** the **Admin SDK Reports API** (Workspace usage/activity reports) and the **Directory API** — for adoption metrics only. **No write scopes are required by this program.**

• **Why required:** Adoption metrics are computed from Workspace usage reports; the program reads them

and never writes user data.

• **How to verify:** In the Admin console (**Account** → **Admin roles**) confirm a role with the

Reports privilege; test the Reports API returns data (see Testing). Reports data can lag ~1–3 days and some metrics require the tenant's edition — **verify in your tenant**.

• **How to implement if missing:** Create a custom admin role limited to **Reports** (and Directory

read) and assign it to the measurement account/service account; enable the **Admin SDK API** in the associated Google Cloud project. Never export or commit a service-account key — use Application Default Credentials (ADC).

6. A champion-network mandate (time allocation)

• **Description:** Agreement from business-unit leaders that nominated **champions** get **allocated

time** (a defined % of their week during their wave) to be trained ahead, support peers, and floor- walk at cutover.

• **Why required:** Champions are the program's distributed workforce; without protected time the

network is nominal and hypercare collapses onto central IT.

• **How to verify:** Confirm BU leaders have agreed a champion ratio and time allocation, and can

nominate names for `Champion Cohort – <BU> – Wave N`.

• **How to implement if missing:** Negotiate the mandate through the sponsor before recruiting

champions.

7. Gemini licensing clarity (per cohort)

• **Description:** A clear record of **which cohorts/waves are licensed for Gemini** (Gemini for

Workspace / Gemini Enterprise add-on) and **which Gemini features are enabled** for them in the Admin console.

• **Why required:** The Gemini learning path is delivered **only** to licensed cohorts, and only for

features that are actually turned on. Gemini feature availability is version- and rollout-dependent.

• **How to verify:** In **admin.google.com** → **Billing / Subscriptions** confirm the Gemini add-on and

seat counts; in **Apps** → **Google Workspace** → **(Gemini/AI settings)** confirm which features are on for which OUs. **Verify in your tenant** — do not assume a feature is available.

• **How to implement if missing:** Confirm licensing with the migration/licensing owner; if a cohort

is unlicensed, omit the Gemini path for that cohort (do not promise features they cannot use).

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The* ***Google Admin console*** *(admin.google.com),* ***Google Cloud / Cloud Identity****, and the* ***Microsoft / Entra*** *sides are authored at exact, current click-by-click detail from the vendors' public documentation.* ***Gemini Enterprise*** *capabilities that are in preview or rolling out are* ***cited or flagged "verify in your tenant"*** *— never invented.* ***Third-party migration tools*** *(e.g. for mail/Drive/Chat data movement) and* ***third-party enterprise apps*** *(ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

Implementation

Follow these steps in order. They implement the house model **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, **pilot-first**. Every artifact is named with the convention `<Purpose> – <Scope> – <Stage>` and that exact name is reused in Testing, Troubleshooting, and the scripts. Production/org-wide actions (launching a wave, sending all-user comms, gating go-live) carry an explicit pass/fail gate and a documented rollback/hold.

> *The five Python tools referenced below live in `scripts/` and are read-only or `--dry-run` by default; pass `--apply` only where a tool supports a write and you have reviewed the dry-run output.*

Phase 1 — Assess

1.1 Confirm the wave schedule and baseline the population

1. Obtain the migration **wave schedule** (cohort → cutover date → user count → OUs/groups) — Req. 1.
2. Confirm each wave maps to real OUs/groups: **admin.google.com** → **Directory** → **Organizational units** and **Directory** → **Groups**.
1. Run the **adoption baseline assessment** to snapshot current Workspace usage and directory readiness **before** any program action, so improvement is measurable later:

```
python scripts/adoption_metrics_report.py --days 30 --out baseline_adoption.csv
```

This reads the Admin SDK Reports API (read-only) and writes a per-app active-usage baseline. Save it as the `Adoption Metrics – Enterprise – v1` baseline.

1.2 Assess audience, roles, and Gemini licensing

1. Segment the population into **roles** (e.g. Frontline, Knowledge Worker, Manager/Exec, IT/Admin) — these become `Learning Path – <Role> – v1`.
1. Confirm **which cohorts are Gemini-licensed** and which Gemini features are enabled (Req. 7); **verify in your tenant** — mark each cohort Gemini-eligible or not.
1. Record the assessment as the input to `Adoption Program – Enterprise – v1`.

Gate A (Assess → Design): wave schedule confirmed, baseline captured, roles + Gemini eligibility recorded. If any is missing, do not proceed.

Phase 2 — Design

2.1 Author the master OCM plan — `Adoption Program – Enterprise – v1`

1. Document, for the whole program: the sponsor and mandate (Req. 2), the awareness → training → reinforcement arc, the champion model, the hypercare model, and the metrics plan.
1. Define **adoption targets** per app (e.g. "≥ X% of a wave active in Gmail within 14 days of cutover") — org-tuned; these become the Validate gates.

2.2 Design the champion network — `Champion Network – Enterprise – v1`

1. Set the **champion ratio** (default ~1 per 50–100 users; tune per Req. 6) and the time allocation.
2. Plan the recruitment, training-ahead, content-equipping, and recognition model.

2.3 Design the comms plan — `Comms Plan – Enterprise – v1`

1. Build the awareness calendar **relative to each wave's cutover date** (e.g. T-21 announce, T-7 "what changes", T-1 "go-live tomorrow", T+0 "you're live / get help", T+7 tips).

1. Map each message to a channel (Req. 3) and note that **pre-cutover** messages go to the **old (M365) mailbox **and** post-cutover **to the** new (Google) mailbox**.

2.4 Design role-based learning paths — `Learning Path – <Role> – v1`

1. For each role, define modules across **Gmail, Drive, Docs/Sheets/Slides, Meet, Chat/Spaces**, sized to what the role does (frontline = lean; knowledge worker = full; manager = delegation/Meet).

1. Add a **Gemini** module **only** to paths for Gemini-eligible cohorts, and only for **enabled** features (Help me write, side-panel, Gems where available) — **verify in your tenant**; label preview features as such.

1. Define the **completion gate**: which modules must be complete before that role's cohort go-live.

2.5 Design the hypercare model — `Hypercare Model – <Wave/Region> – Golive`

1. Size the support surface: floorwalker coverage (default ~1 floorwalker per 75–150 users on day 0, tapering over the hypercare window, tuned per site), a triage/ticket path, and champion escalation.

1. Define the hypercare **exit criteria** (e.g. ticket rate below threshold, adoption trending up).

Gate B (Design → Configure): `Adoption Program – Enterprise – v1` and its sub-plans authored and sponsor-approved.

Phase 3 — Configure

3.1 Build the champion roster and wave plan

1. Collect BU nominations into a roster CSV, then generate the wave-aligned cohorts and coverage check:

```
python scripts/champion_roster_planner.py --roster nominations.csv \
    --waves waves.csv --ratio 75 --out champion_plan.csv
```

This produces `Champion Cohort – <BU> – Wave N` assignments and flags any wave under the target ratio (dry-run/report only — it writes a plan file, changes nothing in the tenant).

1. Fill coverage gaps flagged by the tool before that wave proceeds.

3.2 Stand up the completion-tracking source

1. Configure the LMS/roster (Req. 4) with a course per `Learning Path – <Role> – v1`.
2. Confirm completion can be exported to CSV for the tracker.

3.3 Enable read-only measurement access

1. Create/confirm the least-privilege **Reports**-only admin role (Req. 5) and enable the **Admin SDK API** in the associated Google Cloud project.

1. Authenticate the tooling with **ADC** (`gcloud auth application-default login`, or a service account via domain-wide delegation) — **never** export or commit a key.

1. Smoke-test read access:

```
python scripts/adoption_config_check.py
```

This validates that ADC works, the Reports/Directory APIs are reachable, and **no write scopes** are present (guardrail check) — see Testing.

Gate C (Configure → Pilot): champion plan meets ratio, completion source live, read-only measurement verified.

Phase 4 — Pilot (`Adoption Pilot – Wave 0 – Pilot`)

Run the **entire OCM cycle** on a small, friendly cohort before any production wave.

1. **Awareness:** execute the `Comms Plan – Enterprise – v1` sequence against Wave 0 dates.
2. **Training:** deliver the relevant `Learning Path – <Role> – v1` to the pilot; track completion:

```
python scripts/training_completion_tracker.py --completions lms_export.csv \
    --roster wave0_roster.csv --wave "Wave 0" --out wave0_completion.csv
```

Confirm the completion **gate** is met (per role) before pilot cutover.

1. **Cutover:** the migration program cuts Wave 0 over (this program does not perform the migration).
2. **Hypercare:** staff `Hypercare Model – Wave 0 – Golive` (floorwalkers + champions). Capture tickets.
3. **Analyze themes:** find where users struggled so comms/training can be improved before scale-out:

```
python scripts/hypercare_theme_analyzer.py --tickets wave0_tickets.csv \
    --out wave0_themes.csv
```

1. **Measure:** after ~7–14 days, compare pilot usage to the baseline:

```
python scripts/adoption_metrics_report.py --days 14 --ou "Wave 0" \
    --baseline baseline_adoption.csv --out wave0_adoption.csv
```

Phase 5 — Validate (pilot pass/fail gate)

Evaluate `Adoption Pilot – Wave 0 – Pilot` against the gates (see Testing for exact procedures):

- **Training delivered & gate met** — required modules complete for the pilot roles.
- **Hypercare staffed & coping** — coverage met; ticket rate falling toward exit criteria; top themes identified and mapped back into comms/training fixes.
- **Adoption trending up** — per-app active usage above baseline and moving toward targets.

Rollback / hold: if a gate fails, **do not scale**. Hold the next wave, extend hypercare, re-run the weakest comms/training step for the pilot, apply the theme-analyzer fixes, and re-validate. The program only advances to Expand when Wave 0 passes.

Phase 6 — Expand (production waves)

For **each** production wave, repeat the Pilot cycle with the improvements folded in:

1. Regenerate the wave's `Champion Cohort – <BU> – Wave N` and confirm ratio (`champion_roster_planner.py`).
1. Run `Comms Plan – Enterprise – v1` on the wave's dates.
2. Deliver training; enforce the completion gate (`training_completion_tracker.py`).
3. Cut over (migration program) → staff `Hypercare Model – <Wave/Region> – Golive`.
4. Analyze themes (`hypercare_theme_analyzer.py`) and feed fixes forward to the next wave.
5. Measure adoption vs. baseline/targets (`adoption_metrics_report.py`); publish to `Adoption Metrics – Enterprise – v1`.
1. Apply the **per-wave rollback/hold**: any wave failing its gates pauses further waves until fixed.

Continue until all waves are live, adoption is at/above target, and hypercare has exited to steady-state support (champions + normal service desk).

Testing & Validation

Each procedure below has an explicit **expected pass/fail** signal. Run the Assess/Configure checks before Pilot, and run the Pilot validation gates before scaling to any production wave. All measurement is **read-only**.

T1 — Read-only measurement access works (`adoption_config_check.py`)

Procedure

```
python scripts/adoption_config_check.py
```

Pass: the tool reports ADC is configured, the **Admin SDK Reports** and **Directory** APIs are reachable, and it confirms **no write/OAuth write scopes** are in use (guardrail OK). **Fail:** ADC not found, API not enabled, insufficient role, or a write scope detected. Resolve per Troubleshooting T1 before proceeding — do **not** work around a missing role by granting broad admin.

> *Reports API data can lag ~1–3 days and some counters depend on the tenant edition — **verify in your tenant**; a zero on day-0 is not necessarily a failure.*

T2 — Adoption baseline captured (`adoption_metrics_report.py`)

Procedure

```
python scripts/adoption_metrics_report.py --days 30 --out baseline_adoption.csv
```

Pass: `baseline_adoption.csv` contains per-app active-usage rows (Gmail, Drive, Docs, Meet, Chat, and Gemini where reported) for the window. This is the `Adoption Metrics – Enterprise – v1` baseline. **Fail:** empty output or an auth error → resolve access (T1) and retry.

T3 — Champion coverage meets ratio (`champion_roster_planner.py`)

Procedure

```
python scripts/champion_roster_planner.py --roster nominations.csv \
    --waves waves.csv --ratio 75 --out champion_plan.csv
```

Pass: every wave in `champion_plan.csv` has champions ≥ (wave users ÷ ratio); no wave is flagged `UNDER_RATIO`. Cohorts are named `Champion Cohort – <BU> – Wave N`. **Fail:** one or more waves flagged `UNDER_RATIO` → recruit more champions for those BUs before the wave proceeds (this is a Configure gate).

T4 — Training completion gate met (`training_completion_tracker.py`)

Procedure

```
python scripts/training_completion_tracker.py --completions lms_export.csv \
    --roster wave0_roster.csv --wave "Wave 0" --required-rate 0.90 --out wave0_completion.csv
```

Pass: per-role completion ≥ the required rate (default 90%, org-tuned); the tool prints `GATE: PASS` and lists any stragglers. **Fail:** `GATE: FAIL` for a role → that role's cohort does **not** go live until completion is met (chase stragglers, run a catch-up session). This is the go-live gate.

T5 — Hypercare is staffed and themes are understood (`hypercare_theme_analyzer.py`)

Procedure

```
python scripts/hypercare_theme_analyzer.py --tickets wave0_tickets.csv \
    --expected-floorwalkers 20 --actual-floorwalkers 20 --out wave0_themes.csv
```

Pass: floorwalker coverage meets the `Hypercare Model – Wave 0 – Golive` plan (`STAFFING: OK`) and the tool produces a ranked theme list so the top pain points are known and can be fed back into comms/training. **Fail:** `STAFFING: SHORT` (add floorwalkers/champions) or an unexplained spike in one theme (indicates a comms/training gap to fix before the next wave).

T6 — Adoption is trending up vs. baseline (`adoption_metrics_report.py`)

Procedure (after ~7–14 days post-cutover)

```
python scripts/adoption_metrics_report.py --days 14 --ou "Wave 0" \
    --baseline baseline_adoption.csv --target-rate 0.70 --out wave0_adoption.csv
```

Pass: per-app active-usage for the wave is **above baseline** and moving toward the target; the tool prints `TREND: UP` per app meeting target. **Fail:** flat or below baseline (`TREND: FLAT/DOWN`) → the cohort needs another push (reinforcement comms, targeted training, more champion nudging) before the program treats the wave as done, and **hold** the next wave if the pilot itself is failing.

Pilot validation summary (the scale-out gate)

`Adoption Pilot – Wave 0 – Pilot` **passes** — and the program may advance to **Expand** — only when **T4 (training gate)**, **T5 (hypercare staffed, themes understood)**, and **T6 (adoption trending up)** all pass for Wave 0. Any failure triggers the documented **hold/rollback**: pause the next wave, remediate, re-validate.

Troubleshooting

Common failure modes across the program and its tooling, with resolutions. Test IDs (T1–T6) refer to Testing; artifact names use the `<Purpose> – <Scope> – <Stage>` convention.

T1 — `adoption_config_check.py` reports FAIL

- **"no ADC" / no Application Default Credentials.** The account isn't authenticated. Run

`gcloud auth application-default login` as an account holding the least-privilege **Reports-only** admin role, or configure a service account via domain-wide delegation. **Never** export or commit a service-account key to make this pass.

- **"Reports API not reachable / not enabled / insufficient role."** Enable the **Admin SDK API** in

the associated Google Cloud project, and confirm the account has the **Reports** privilege (**admin.google.com** → **Account** → **Admin roles**). Wait a few minutes after enabling.

- **"WRITE scope(s) detected."** Something granted more than read-only. Re-authenticate ADC with only

the read-only scopes; do not proceed with a write-capable credential — the program is read-only by design.

- **"missing libs."** `pip install google-api-python-client google-auth`.

T2 — `adoption_metrics_report.py` returns empty / zeros

- **Data lag.** The Reports API can lag ~1–3 days; the tool already walks back from `today-2`. A very

recent window may legitimately have no data — widen `--days` or wait a day. **Verify in your tenant.**

- **Edition/feature gating.** Some counters (and **Gemini** usage in particular) depend on the tenant

edition and rollout — a `None`/blank for an app can mean "not reported for this edition", not "no usage". Confirm which metrics your edition exposes before treating a zero as a finding.

- **Auth error.** Resolve via T1 first, then retry.

T3 — A wave is flagged `UNDER_RATIO` (`champion_roster_planner.py`)

• **Genuine shortfall.** Recruit more champions for the flagged `Champion Cohort – <BU> – Wave N` before that wave proceeds — this is a Configure gate (Req. 6). Do not lower the ratio just to pass.

• **Data mismatch.** Confirm `business_unit` and `wave` strings match **exactly** between `nominations.csv` and `waves.csv` (whitespace/casing differences split cohorts). Normalize the CSVs and re-run.

• **Nominal champions.** Champions without the agreed **time allocation** shouldn't count — validate the mandate before including them.

T4 — Training gate fails (`training_completion_tracker.py` → `GATE: FAIL`)

• **Real stragglers.** The tool lists them — run a catch-up session / targeted reminders; the cohort (for that role) does **not** go live until the gate is met.

• **Status values not recognized.** The tracker counts statuses like *complete/completed/pass/done*. If your LMS exports a different label, map it to one of those in the export before running.

• **Email mismatch.** Completion is joined on `email` (case-insensitive). If the LMS uses aliases or display names, normalize to the primary Google address in both files.

T5 — Hypercare overwhelmed or `STAFFING: SHORT`

• `STAFFING: SHORT`**.** Add floorwalkers or pull in `Champion Cohort – <BU> – Wave N` members to meet the `Hypercare Model – <Wave/Region> – Golive` plan; front-load day-0 coverage.

• **One theme dominates** (`hypercare_theme_analyzer.py`). A single large theme (e.g. "missing mail", "sign-in") signals a **comms or training gap** — fix that specific message/module and feed it forward to the next wave; don't just add support headcount to a preventable problem.

• **"Other" is the biggest bucket.** The keyword taxonomy in the analyzer needs tuning for your environment — add the recurring terms you see to the `THEMES` map and re-run.

T6 — Adoption flat or below baseline (`TREND: FLAT/DOWN`)

• **Too soon.** Usage often dips right after cutover before recovering; measure at ~7–14 days, not day 1, and always **vs. the baseline** you captured in Assess.

• **Reinforcement gap.** A persistently flat cohort needs another push — reinforcement comms, a targeted refresher for the weak app, and champion nudging — before the wave is treated as done.

• **Wrong tool still in use.** If users keep using the old M365 tool, check whether coexistence/routing

is inadvertently keeping the old path alive (that's a migration-side issue to escalate) and whether comms clearly told users to stop using the old tool.

• **Pilot failing.** If `Adoption Pilot – Wave 0 – Pilot` won't trend up after remediation, **hold** the next wave and revisit the OCM plan — do not scale a program that isn't working.

Program-level issues

• **Sponsor gone quiet.** Champions and mandatory-training expectations lose authority. Re-secure the sponsor's per-wave leadership message before the next wave (Req. 2) — a program without visible sponsorship should not advance to Expand.

• **Comms not landing.** Confirm pre-cutover messages go to the **old (M365)** mailbox and post-cutover to the **new (Google)** mailbox, and that per-wave distribution groups resolve to the right cohort (Req. 3).

• **Gemini content promised but unavailable.** Only deliver the Gemini path to **licensed** cohorts and only for **enabled** features; if a feature isn't on for that OU, remove it from the path — **verify in your tenant** (Req. 7). Promising an unavailable feature erodes trust in the whole program.

Best Practices

Durable practices for running the adoption, change-management, and training program at enterprise scale. These sustain adoption **after** the migration team has moved on.

Program & change management

• **Lead with a real sponsor, not a logo.** The `Adoption Program – Enterprise – v1` mandate must be visibly carried by the executive sponsor at every wave — a leadership message at each go-live beats any amount of IT email. If the sponsor goes quiet, the program stalls; re-engage before scaling.

• **Align everything to the wave, not the calendar.** Time awareness, training, and hypercare **relative to each cohort's cutover date**, so no cohort trains three weeks early (and forgets) or the day after (too late).

• **Make it about the job, not the tool.** Frame change as "how you'll do *your* work" per role — not "we're moving to Google". Role-based `Learning Path – <Role> – v1` content should use the audience's real tasks.

• **Pilot honestly.** `Adoption Pilot – Wave 0 – Pilot` exists to *find problems*. Pick a friendly but representative cohort, and treat a failed gate as success (you caught it before 150k users).

Champion network

• **Protect champion time.** A champion without allocated time (Req. 6) is a name on a slide. Confirm the time mandate per `Champion Cohort – <BU> – Wave N` before counting them toward coverage.

• **Train champions ahead of their cohort.** They must be fluent *before* their business unit cuts over so they can floor-walk credibly.

• **Recognize them.** Visible recognition (sponsor shout-outs, a badge, manager awareness) keeps the network alive across many waves; unrecognized champions quietly stop.

• **Close the loop with champions' field intelligence.** They see pain first — route their feedback into `Comms Plan – Enterprise – v1` and `Learning Path – <Role> – v1` updates each wave.

Training

• **Just-in-time, bite-sized, multi-modal.** Short micro-learning + quick-reference guides + a few live sessions beat one long course. Record live sessions for shift workers and stragglers.

• **Gate go-live on completion, and mean it.** The 90%-per-role gate (T4) is a genuine hold — waiving it teaches the org the gate is optional.

• **Teach Gemini only where it's licensed and enabled.** Deliver the Gemini module **only** to Gemini-eligible cohorts and only for **turned-on** features; **verify in your tenant** and label preview features. Promising a feature a cohort can't use erodes trust in the whole program.

Hypercare & support

• **Front-load floorwalkers on day 0, then taper.** The first 48–72 hours generate most tickets; staff `Hypercare Model – <Wave/Region> – Golive` heavy early and taper against real ticket-rate data, not a fixed date.

• **Define hypercare exit criteria up front** (ticket rate + adoption trend), so the transition to steady-state support (champions + normal service desk) is evidence-based, not arbitrary.

• **Mine ticket themes every wave** (`hypercare_theme_analyzer.py`) and *act on the top three* before the next wave — the same confusion should not recur wave after wave.

Measurement & data hygiene

• **Measure read-only, always.** The program **never** writes user data to measure it; keep tooling on ADC with a **Reports-only** role and run `adoption_config_check.py` to prove no write scopes crept in.

• **No secrets in tooling.** Never export or commit service-account keys; use ADC / the environment. Treat any key file as an incident.

• **Baseline before you touch anything**, then always report **vs. baseline** — "up from where we started" is the honest adoption story and the ROI evidence the sponsor needs.

• **Respect the data lag and edition caveats.** Reports API can lag ~1–3 days and some counters depend

on edition/features — **verify in your tenant** before declaring a trend; don't over-react to a single day.

Sustaining adoption

- **Hand off, don't drop.** When hypercare exits, transfer the champion network and the

`Adoption Metrics – Enterprise – v1` dashboard to a steady-state owner so usage keeps being watched.

- **Keep a reinforcement drumbeat.** Periodic tips, "new feature" comms (especially as Gemini features

roll out), and refreshers stop the post-cutover usage dip from becoming permanent.

Deliverables Summary

This solution delivers a complete, self-guided **adoption, change-management, and training program** for an enterprise Microsoft 365 → Google Workspace transformation, plus five Python tools that instrument and operate it against real Google APIs (read-only / dry-run by default).

Documentation

- **Solution Overview** — the OCM model: sponsor, champion network, the

awareness → training → reinforcement arc, role-based learning paths, hypercare/floorwalking, and adoption measurement; the wave-aligned reference design and named artifacts.

- **Requirements Guide** — the solution-specific prerequisites (wave schedule, exec sponsorship, comms

channels, training/LMS capacity, read-only measurement access, champion mandate, Gemini licensing), each with verify/implement steps, building on the shared `global-requirements-google.md`.

- **Implementation Guide** — ordered steps across Assess → Design → Configure → Pilot → Validate →

Expand: build the champion network, run the comms/awareness campaign, deliver role-based training, stand up hypercare, and measure adoption — pilot-first, with pass/fail gates and rollback/hold.

- **Testing & Validation** — six procedures (T1–T6), each with an explicit pass/fail, culminating in

the pilot scale-out gate.

- **Troubleshooting** — common failure modes (access, measurement, coverage, completion, hypercare,

adoption trend) and their resolutions.

- **Best Practices** — durable practices for sponsorship, champions, training, hypercare, measurement

hygiene, and sustaining adoption after cutover.

- **Executive Summary** — the outcome, approach, and why it is safe/governed.

Python tools (`scripts/`) — read-only or `--dry-run` by default

1. `adoption_metrics_report.py` — *Assessment / report generator.* Reads the **Admin SDK Reports

API **(Workspace usage) read-only to compute per-app active-usage (Gmail, Drive, Docs, Meet, Chat, Gemini where reported), optionally per OU/wave and** vs. a baseline**, and prints a `TREND` signal. Produces the adoption baseline and per-wave adoption reports. No writes.

1. `champion_roster_planner.py` — *Scope/config helper.* Turns BU champion nominations + the wave

schedule into wave-aligned `Champion Cohort – <BU> – Wave N` assignments, checks coverage against a target ratio, and flags any wave `UNDER_RATIO`. Writes a plan file only — changes nothing in the tenant.

1. `training_completion_tracker.py` — *Validation / gate.* Joins an LMS/roster completion export to

a wave roster, computes per-role completion, applies the go-live completion gate (`GATE: PASS/FAIL`), and lists stragglers. Read-only over its inputs.

1. `hypercare_theme_analyzer.py` — *Guardrail / analysis.* Analyzes hypercare ticket exports into

ranked themes and checks floorwalker staffing against the hypercare plan (`STAFFING: OK/SHORT`), so the top pain points feed back into comms/training. Read-only.

1. `adoption_config_check.py` — *Policy/guardrail check.* Verifies the measurement setup is

safe: ADC present, Reports/Directory APIs reachable, and **no write scopes** in use — the read-only guardrail for the whole program. No writes.

Safety: every tool is safe-by-default (read-only or `--dry-run` unless `--apply` is passed where supported), uses **Application Default Credentials** / the environment, and contains **no hardcoded credentials, service-account keys, or secrets**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

adoption_config_check.py

adoption_metrics_report.py

champion_roster_planner.py

hypercare_theme_analyzer.py

training_completion_tracker.py